

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 1: 1. Описание модели улучшений (Пишем в самом верху, вне экрана)

```
// Создаем структуру данных для улучшений нашей подводной лодки.  
// Пишем этот класс в самом верху файла, за пределами всех функций.  
data class SubUpgrade(  
    val id: String,  
    val name: String,  
    val level: Int,  
    val baseCost: Long,  
    val icon: String  
) {  
    // Рассчитываем стоимость улучшения в зависимости от его уровня  
    val cost: Long  
        get() = baseCost + (level * (baseCost * 0.6)).toLong()  
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 2: 2. Элемент интерфейса: Карточка Улучшения (SubUpgradeRow)

```
// Отдельный элемент UI для одной строки в магазине улучшений.  
// Помогает сделать код главного экрана проще и избежать лишних скобок.  
@Composable  
fun SubUpgradeRow(  
    upgrade: SubUpgrade,  
    canAfford: Boolean,  
    onUpgradeClick: () -> Unit  
) {  
    Card(  
        modifier = Modifier.fillMaxWidth(),  
        colors = CardDefaults.cardColors(containerColor = Color(0xFF0F1C3F))  
    ) {  
        Row(  
            modifier = Modifier.fillMaxWidth().padding(10.dp),  
            horizontalArrangement = Arrangement.SpaceBetween,  
            verticalAlignment = Alignment.CenterVertically  
        ) {  
            Column {  
                Text(upgrade.icon + " " + upgrade.name, color = Color.White, fontWeight =  
                    FontWeight.Bold)  
                Text("Уровень: " + upgrade.level, color = Color(0xFF4FC3F7), fontSize = 12.sp)  
            }  
            Button(  
                onClick = onUpgradeClick,  
                enabled = canAfford,  
                colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF00E676))  
            ) {  
                Text(upgrade.cost.toString() + " ⚙", fontSize = 12.sp, color = Color.Black)  
            }  
        }  
    }  
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 3: 3. Объявление главного экрана (Конструкция)

```
// Начинаем создавать главный экран игры.  
// Весь последующий код до шага с превью будет находиться внутри этой функции.  
@Composable  
fun SubmarineExplorerScreen() {  
    // Внимание: проверьте наличие закрывающей фигурной скобки '}' в самом конце  
    // файла!
```

ЭТАП 4: 4. Состояние игры (Переменные)

```
// Вставляем этот код внутрь функции SubmarineExplorerScreen.  
// Эти переменные хранят текущие параметры нашего батискафа.  
var gold by remember { mutableStateOf(100L) }  
var depth by remember { mutableStateOf(0f) }  
var oxygen by remember { mutableStateOf(100f) }  
var isDiving by remember { mutableStateOf(false) }  
  
// Уровни систем подводной лодки  
var hullLevel by remember { mutableStateOf(1) }  
var tankLevel by remember { mutableStateOf(1) }  
var engineLevel by remember { mutableStateOf(1) }
```

ЭТАП 5: 5. Математические расчеты (Динамические параметры)

```
// Вычисляем максимальные возможности батискафа на основе уровней прокачки.  
val maxOxygen = tankLevel * 100f  
val maxDepth = hullLevel * 200f  
val diveSpeed = engineLevel * 6f  
val oxygenUsageRate = 3f / tankLevel // С баками высокого уровня расход меньше
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 6: 6. Игровой цикл погружения (LaunchedEffect)

```
// Запускаем фоновый таймер погружения при активации флага isDiving.
LaunchedEffect(isDiving) {
    if (isDiving) {
        while (isDiving) {
            delay(500) // Пауза полсекунды между тиками

            // Тратим кислород
            oxygen = (oxygen - oxygenUsageRate).coerceAtLeast(0f)

            if (oxygen <= 0f) {
                // Авария: кислород закончился, лодка экстренно возвращается
                isDiving = false
                depth = 0f
                gold = (gold - 50L).coerceAtLeast(0L) // Штраф за спасение
            } else if (depth < maxDepth) {
                // Погружаемся и собираем золото за глубину
                depth = (depth + diveSpeed).coerceAtMost(maxDepth)
                gold += (depth / 30).toLong()
            }
        }
    }
}
```

ЭТАП 7: 7. Главный макет экрана (Контейнеры Box и Column)

```
// Настраиваем фон под цвет глубокого океана и создаем основной вертикальный
// список.
Box(
    modifier = Modifier.fillMaxSize().background(Color(0xFF050E1E)),
    contentAlignment = Alignment.TopCenter
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        /* Сюда мы добавим остальные визуальные элементы в следующих шагах */
    }
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 8: 8. Информационная панель (Метрики батискафа)

```
// Вставляем этот блок внутрь контейнера Column (Шаг 7)
Text("ПОДВОДНАЯ МИССИЯ БЕЗДНА ", color = Color.White, fontSize = 18.sp,
fontWeight = FontWeight.Bold)

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
Text("Глубина: ${depth.toInt()} / ${maxDepth.toInt()} м", color =
Color(0xFF4FC3F7))
    Text("Золото: $gold 🍊", color = Color(0xFFFFD54F))
}

// Панель кислорода
Column(modifier = Modifier.fillMaxWidth()) {
Text("Запас кислорода: ${oxygen.toInt()}/${maxOxygen.toInt()}", color =
Color.LightGray, fontSize = 12.sp)
    Spacer(modifier = Modifier.height(4.dp))
    LinearProgressIndicator(
        progress = { oxygen / maxOxygen },
        modifier = Modifier.fillMaxWidth().height(10.dp),
        color = if (oxygen < 30f) Color.Red else Color(0xFF00E676)
    )
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 9: 9. Панель управления (Кнопки действий)

```
// Добавляем кнопки под индикаторами (внутри Column из Шага 7)
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(12.dp)
) {
    Button(
        onClick = { isDiving = !isDiving },
        modifier = Modifier.weight(1f),
        colors = ButtonDefaults.buttonColors(
containerColor = if (isDiving) Color(0xFFD32F2F) else Color(0xFF0288D1)
        )
    ) {
        Text(if (isDiving) "ВСПЛЫТИЕ" else "ПОГРУЖЕНИЕ")
    }

    Button(
        onClick = {
            oxygen = maxOxygen
            depth = 0f
        },
        modifier = Modifier.weight(1f),
        enabled = !isDiving,
        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF455A64))
    ) {
        Text("ЗАПРАВКА")
    }
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 10: 10. Список улучшений (LazyColumn)

```
// Список апгрейдов, использующий наш созданный ранее шаблон SubUpgradeRow.
// Размещаем его внизу основного контейнера Column.
val upgrades = listOf(
    SubUpgrade("hull", "Прочность корпуса", hullLevel, 40L, "♥ "),
    SubUpgrade("tank", "Кислородные баллоны", tankLevel, 50L, " "),
    SubUpgrade("engine", "Турбо-двигатель", engineLevel, 70L, " ")
)

LazyColumn(
    modifier = Modifier.fillMaxWidth().weight(1f),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(upgrades) { item ->
        SubUpgradeRow(
            upgrade = item,
            canAfford = gold >= item.cost && !isDiving,
            onUpgradeClick = {
                gold -= item.cost
                when (item.id) {
                    "hull" -> hullLevel++
                    "tank" -> {
                        tankLevel++
                        oxygen = tankLevel * 100f
                    }
                    "engine" -> engineLevel++
                }
            }
        )
    }
}
```

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 11: 11. Проверка внешнего вида экрана (Preview)

```
// Этот блок пишется СНАРУЖИ функции SubmarineExplorerScreen.  
// Проверь, что закрыл все фигурные скобки экрана перед добавлением превью.  
@Preview  
@Composable  
fun SubmarineExplorerPreview() {  
    MaterialTheme {  
        SubmarineExplorerScreen()  
    }  
}
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь Систему Сонара: создай переменную sonarActive. Сделай так, чтобы при включенном сонаре (расходует дополнительные 1.5 единиц кислорода за тик) с шансом 20% (используй `Random.nextFloat() < 0.20f`) игрок находил дополнительное золото (+100 монет) во время погружения.